

Supplementary code: generating a logistic-regression sample with a known population R-squared

Purpose

This file illustrates, for a single case – a logistic regression whose population R-squared is exactly known in advance – the calibrate-then-sample method used throughout the simulations reported in the paper. The code below is a stripped-down extract of the logistic branch of `sample_by_r2()` from the `Rsimcity` package (<https://github.com/mcfanda/RsimCity>): input validation, caching, and the branches for the other model families (gaussian, multinomial, ordinal) and for targeting a single predictor's eta-squared rather than the whole model's R-squared have all been removed, so that only the logic essential to this one case remains. All of those variations, their defaults, and their full implementations can be found in the package itself.

The method in three steps

1. Build a numerical-integration grid over the joint distribution of the k predictors, using Gauss-Hermite quadrature.
2. Find, by root-finding, the coefficient scale that makes the *population* R-squared – computed exactly on that grid – equal the requested target.
3. Only then draw an actual finite sample of size n from the calibrated population.

Code

```
sample_logistic_r2 <- function(n, R2, k = 1, beta = rep(1, k),
                              intercept = 0, x_mean = rep(0, k), x_cov = diag(k),
                              order = 10, seed = NULL) {

  ## Gauss-Hermite quadrature grid over the k predictors
  ## using statmod::gauss.quad.prob(). It returns nodes/weights for a standard normal density,
  ## with weights already summing to 1, so that for any function f, E[f(X)] = sum(w * f(Xgh)).
  ## The grid is then transformed through the Cholesky factor of x_cov and shifted by x_mean, giving qu
  ## for the actual (correlated, possibly non-centered) predictor distribution.
  gh <- statmod::gauss.quad.prob(order, dist = "normal")
  g <- as.matrix(expand.grid(rep(list(seq_len(order)), k)))
  Z <- matrix(gh$nodes[g], ncol = k)
  w <- apply(matrix(gh$weights[g], ncol = k), 1, prod)
  Xgh <- sweep(Z %*% chol(x_cov), 2, x_mean, "+")

  ## Population deviance R-squared, as a function of a coefficient scale `a`
  ## beta = a * beta direction; the model's probability at every grid node gives the
  ## deviance-based R-squared exactly (no simulation is involved in this computation).
  r2_of_a <- function(a) {
    eta <- intercept + drop(Xgh %*% (a * beta))
    prob <- stats::plogis(eta)

    p <- cbind(1 - prob, prob)
    p <- pmax(p, .Machine$double.eps)
    p0 <- colSums(p * w) # population marginal probabilities
  }
```

```

D1  <- -2 * sum(w * rowSums(p * log(p)))      # model deviance
D0  <- -2 * sum(p0 * log(p0))                  # null-model deviance
1 - D1 / D0
}

## ---- 3. Root-find the coefficient scale that produces the target R2 -----
upper <- 1
while (r2_of_a(upper) < R2 && upper < 1e6) upper <- upper * 2
a <- stats::uniroot(function(a) r2_of_a(a) - R2, c(0, upper))$root
beta_final <- a * beta

## ---- 4. Draw the actual finite sample from the calibrated population -----
if (!is.null(seed)) set.seed(seed)
X <- sweep(matrix(stats::rnorm(n * k), n, k) %%% chol(x_cov), 2, x_mean, "+")
colnames(X) <- paste0("x", seq_len(k))
eta <- intercept + drop(X %%% beta_final)
y  <- stats::rbinom(n, 1, stats::plogis(eta))

data.frame(y = y, X)
}

```

Example: a single predictor, R-squared = .10

```

dat <- sample_logistic_r2(n = 2000, R2 = .10, seed = 1)
mod <- glm(y ~ x1, data = dat, family = binomial())
1 - mod$deviance / mod$null.deviance      # sample deviance R2, close to the .10 target

## [1] 0.09601619

```

Example: three correlated predictors, R-squared = .20

```

Sigma <- matrix(c(1, .3, .2,
                  .3, 1, .4,
                  .2, .4, 1), nrow = 3)

dat3 <- sample_logistic_r2(n = 5000, R2 = .20, k = 3, x_cov = Sigma, seed = 1)
mod3 <- glm(y ~ x1 + x2 + x3, data = dat3, family = binomial())
1 - mod3$deviance / mod3$null.deviance    # close to the .20 target

## [1] 0.1916439

```